

Document 16 — System Design & Architecture

This is for self-study and live mentoring. Full plan: [CURRICULUM.md](#)

Prerequisites

[14_debugging_lab](#) (you need the failure vocabulary from it), [15_five_projects_index](#)

How to Read & Practice This Document

- **What:** turning an unclear business request into a design you can actually defend, and defending your trade-offs when pushed.
- **Why:** this is the skill that separates "can build a feature" from "can be trusted to own a whole system" — and it's what a senior-level interview actually tests.
- **When:** any time you're handed a vague request instead of a clear spec.
- **How to practice:** for each prompt below, write your design **before** discussing it with me — I review it like a senior architect afterward, I don't hand you the design first.

The Story — what this document is actually building

Every document before this one hands you a fairly clear spec to build against. This one hands you almost nothing — sometimes just one sentence, sometimes a few people wanting different things — and asks you to turn that into a real system design, the way it actually happens at a job. Nobody hands a new hire a finished spec for an AI feature. They say "can we build a bot that answers customer questions," or three stakeholders each want something slightly different, and turning that fog into an actual architecture — which agents, what they do, what happens when something breaks — is the job.

That's why the 3 exercises below escalate the way they do: - **Basic** — one clean sentence, one agent. Practice not overbuilding, and asking the couple of questions that actually matter first. - **Intermediate** — a clean request needing several agents in a specific order. Same matching skill, plus picking the right handoff pattern and deciding what happens when a step fails. - **Real-world** — the "clean" part is gone. Different people want different, conflicting things. Spotting the conflicts and asking about them comes before any design.

Everything you produce maps onto the "What You'll Design (for each prompt)" checklist below — same categories every time, just harder to fill in cleanly as requirements get messier. And the whole

thing is rehearsal for "Interview Topics Preview" further down: a real system-design interview is a vague prompt, your design, pushback, and you defending or revising it. Do all three exercises, and that format stops being intimidating.

Jump to: [Core Concepts](#) · [Practice Exercises](#)

Core Concepts (read this first — everything you need is here)

Unclear requirements are the real test

A vague request ("build a customer support bot") isn't a spec with details left out — it's the actual starting point. **Why treating it as "just fill in the blanks" is the wrong instinct:** the real skill being tested is *asking the right questions before designing anything* — what happens if this gets it wrong? how much traffic is expected? what data can it access? A design built on silent guesses about those answers is fragile in exactly the ways a design built on clearly stated assumptions isn't — because at least those assumptions are visible, and can be challenged.

Requirements → architecture is matching, not inspiration

Every design choice should trace back to a specific requirement, not to "what's a popular pattern right now." **How to actually do this:** for every piece you propose (one specific agent, a database choice, a caching layer), you should be able to name the requirement it exists to satisfy. If you can't, it's probably added complexity that isn't needed yet (an overbuilt multi-agent system for a task one agent could handle) or a fix for a problem you don't have yet (a caching layer for traffic that doesn't exist yet).

Non-functional requirements shape the design just as much as functional ones

"What should it do" (functional) is usually somewhat clear from the request. "How fast, how reliable, how much can it cost, how many users" (non-functional) is what actually decides the architecture — a support bot for 50 people inside a company, and a support bot for 500,000 members of the public, are the *exact same functional request* with completely different correct designs. **Why this is the part beginners skip and experienced engineers don't:** scale, cost, and reliability targets decide whether you need a queue, whether work should happen right away or later in the background, and whether one agent or a spread-out multi-agent system is even worth it — the functional request alone can't tell you any of that.

Defending a design when it's pushed on

Being asked "why not X instead" isn't a sign your design is wrong — it's exactly the format of a senior-level review. **How to handle it well:** repeat back the trade-off you actually weighed (not just "because I picked it"), explain what X would cost you specifically for *this* system's requirements, and be willing to say "actually, you're right, X is better here" when it genuinely is. Defending a choice out

of reflex, no matter how good the pushback is, looks worse than updating your view with a clear reason.

Go Deeper (Optional)

You don't need any of these to understand the Core Concepts above — use them if you want a second explanation or more detail. - [system-design-primer \(GitHub\)](#) — the standard general system-design reference; read the "how to approach a system design interview problem" section closely. - Read [Anthropic — Building Effective Agents](#) again — the patterns from Doc11 are your building blocks here, not new theory.

Practice Exercises

Why only 3 here, not 5: this document's real practice is live — you design, I review like a senior architect, we go back and forth. These 3 give you a fixed starting point at increasing difficulty; the actual depth comes from the conversation after each one, not from more written exercises.

Jump to a design prompt: [Basic](#) · [Intermediate](#) · [Real-world](#)

Basic — a simple single-agent design

- **What:** design (on paper) a single-agent customer-support bot, given only: "answer questions about our product docs, hand off to a human if unsure."
- **Why:** this checks whether you can translate a one-sentence request into a real design without over-building it — the temptation at this size is to add unneeded complexity.
- **When you'll hit this for real:** literally the first design conversation of almost any new AI feature at a real job — someone gives you one sentence, you have to ask the right questions and produce a design.
- **How to do it:** write down your assumptions first (what happens on a wrong answer? what volume?), then the design: agent, tools, routing, and where it hands off to a human.
- **Stuck?** [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

Intermediate — a multi-agent pipeline design

- **What:** design a multi-agent content pipeline given: "research a topic, draft an article, fact-check it, only publish if fact-check passes."
- **Why:** this is a direct rehearsal of Project 4's actual shape — designing it fresh, from requirements, is different from following steps that already decided the shape for you.
- **When you'll hit this for real:** any "build me a pipeline" request with clearly ordered phases — this is an extremely common real request shape.

- **How to do it:** identify the phases first, decide which pattern from Doc11 fits (sequential? does fact-check need to loop back to draft?), then design agents/state/routing around that decision.
- **Stuck?** [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

Real-world — unclear, conflicting requirements

- **What:** design a system given unclear, conflicting requirements from different people (a realistic, messy prompt).
- **Why:** real requirements are never as clean as a training exercise — this is where the actual senior skill (asking the right clarifying questions before designing) gets tested for real.
- **When you'll hit this for real:** almost every real system-design conversation you'll ever have — stakeholders rarely agree with each other up front.
- **How to do it:** before designing anything, write down every place the requirements conflict or are unclear, and what question you'd ask to resolve each one — then design around your best guess, flagging the assumption.
- **Stuck?** [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

What You'll Design (for each prompt)

- Agents (how many, which pattern from Doc11, and why)
- Tools
- State (shared vs. private)
- Database
- APIs
- RAG (if it applies)
- Routing logic
- Security
- Scaling plan
- Watching/monitoring plan

Expected Behavior

Your designs should already account for the failure types from [14_debugging_lab](#), without being asked to — for example, you should already be asking "what stops this from looping forever" before I have to ask you.

Interview Topics Preview

A full system-design interview run-through — this document *is* the interview-prep format (question → your design → my review → correction), just done at the scale of a whole system.

Move On When

Your designs already account for the failure types from Doc14, without me prompting for them, and you can defend at least one design choice when pushed on it directly. Full details:

[CURRICULUM.md §4](#).

Stuck on where to even start a design? Ask for **Hint 1** (what question to ask about the requirements first) through **Hint 4**. Ask for a full design only if you say "**Show me the solution.**"